Falcon-512 Verification on Starknet

Implementation Plan, Mathematics, and Optimization

`falcon-starknet`

July 6, 2026

Abstract

We describe a from-scratch implementation of **Falcon-512** (NIST FN-DSA) signature *verification* for Starknet/Cairo. Signing and key generation use floating-point Gaussian sampling and remain off-chain; only verification—which is integer-only—runs on-chain. We give the mathematics of each stage (modular arithmetic, the negacyclic number-theoretic transform, SHAKE-256 hash-to-point, signature/key decoding, and the norm-bounded acceptance test), the data flow between modules, the code-generation pipeline that emits an unrolled NTT, and the optimization work (lazy modular reduction, the felt-shift trick, and the compiler limits that shape the design). All figures are drawn with Ti*k*Z.

Contents

1 Overview and repository layout

The system is a vertical of independently validated layers. A Rust *reference* (Pornin's public-domain `fn-dsa`) generates genuine signatures and known-answer fixtures; a Rust *code generator* emits an unrolled Cairo NTT; and the Cairo *verifier* consumes the fixtures. Figure 1 shows the module dependencies.

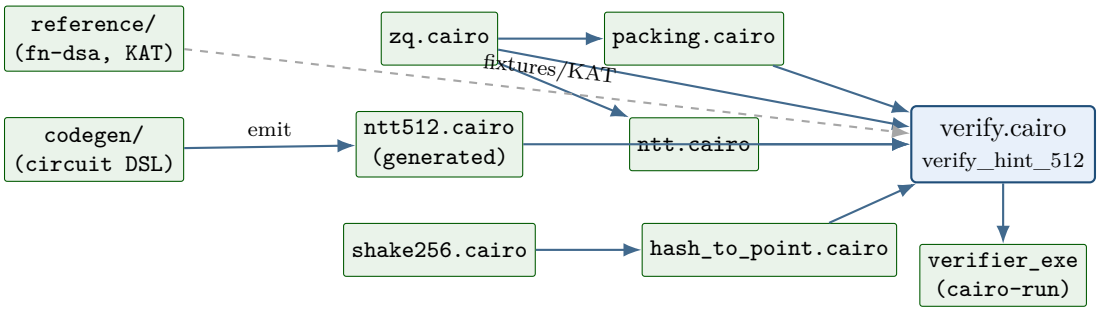

Figure 1: Module dependency flow. Solid: Cairo compile/use edges; dashed: off-chain fixtures. `ntt512.cairo` is emitted by the code generator; `verify.cairo` composes the transform, ring arithmetic, packing, and the norm test.

2 Mathematical preliminaries

Falcon operates over the ring

$$\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1), \qquad n = 512, \quad q = 12289 = 3 \cdot 2^{12} + 1.$$

Since $2n = 1024 \mid q - 1$, $\mathcal{R}_q$ admits a length-n negacyclic NTT. A key pair is a short basis of an NTRU lattice; the public key is $h \in \mathcal{R}_q$. A signature of a message m is a pair of short polynomials (s_1, s_2) together with a 40-byte nonce r, satisfying

$$s_1 + s_2\, h \equiv c \pmod{q,\ x^n + 1}, \qquad c = \mathrm{HashToPoint}(r \| m) \in \mathcal{R}_q. \tag{1}$$

Verification recomputes c, recovers $s_1 = c - s_2 h$, and accepts iff the pair is short:

$$\|(s_1, s_2)\|^2 \;=\; \sum_{i=0}^{n-1} \big(\langle s_{1,i}\rangle^2 + \langle s_{2,i}\rangle^2\big) \;\le\; \lfloor \beta^2 \rfloor = 34{,}034{,}726, \tag{2}$$

where $\langle x \rangle$ is the *centered* representative, $\langle x \rangle = x$ if $x \le \lfloor q/2 \rfloor$ and $x - q$ otherwise, so $\langle x \rangle \in [-\lfloor q/2 \rfloor, \lfloor q/2 \rfloor]$.

3 Per-process mathematics

3.1 Base ring arithmetic ($\mathbb{Z}_q$)

Coefficients are elements of $\mathbb{Z}_q$, represented canonically in $[0, q)$. The core operations are

$$\mathrm{add}(a, b) = (a + b) \bmod q, \quad \mathrm{sub}(a, b) = (a + q - b) \bmod q, \quad \mathrm{mul}(a, b) = (a \cdot b) \bmod q.$$

On Cairo we track ranges in the type system, $\mathbb{Z}_q \cong \texttt{BoundedInt}\langle 0, q{-}1\rangle$, so that overflow checks are discharged at compile time (Section 6).

3.2 Negacyclic number-theoretic transform

Let ψ be a primitive $2n$-th root of unity mod q (so $\psi^n \equiv -1$); we use $\psi = g^{(q-1)/2n}$ with generator $g = 11$. The forward transform is

$$\hat{a}_j \;=\; \sum_{i=0}^{n-1} a_i\, \psi^{\,i(2j+1)} \pmod q, \qquad j = 0, \ldots, n-1. \tag{3}$$

Equivalently we evaluate the polynomial $a(x)$ at the n roots of $x^n + 1$, namely $\{\psi^{2j+1}\}$. In practice (3) is computed by a Cooley–Tukey decimation with $\zeta_k = \psi^{\mathrm{brv}(k)}$ (bit-reversed powers) in $\log_2 n$ layers of butterflies

$$(u, v) \;\mapsto\; \big(u + \zeta v,\; u - \zeta v\big) \pmod q. \tag{4}$$

The transform diagonalizes negacyclic convolution: writing $a \circledast b$ for the product in $\mathcal{R}_q$,

$$\mathrm{NTT}(a \circledast b) \;=\; \mathrm{NTT}(a) \odot \mathrm{NTT}(b) \qquad \text{(pointwise)}, \tag{5}$$

which is the identity every verifier variant relies on. The inverse INTT satisfies $\mathrm{INTT}(\mathrm{NTT}(a)) = a$ and carries the n^{-1} scaling.

3.3 Hash-to-point and Keccak-$f[1600]$

FN-DSA fixes the challenge map as a SHAKE-256 extendable-output function. For a raw message with empty context the absorbed string is

$$\text{HashToPoint}: \quad \mathrm{absorb}\big(\, r \,\|\, \mathrm{hpk} \,\|\, \texttt{0x00} \,\|\, \texttt{0x00} \,\|\, m \,\big), \qquad \mathrm{hpk} = \mathrm{SHAKE256}(pk)_{[0:64]}. \tag{6}$$

The squeeze phase is read as a stream of 16-bit big-endian words w; each is accepted by rejection sampling

$$w < 5q \;\Rightarrow\; \text{coefficient} \;= w \bmod q, \qquad (5q = 61445), \tag{7}$$

until n coefficients are produced. SHAKE-256 is the sponge over the 1600-bit Keccak permutation with rate $r = 1088$ bits and pad byte `0x1F`. One permutation is 24 rounds of

$$\theta \;\to\; \rho \;\to\; \pi \;\to\; \chi \;\to\; \iota,$$

operating on a 5×5 array of 64-bit lanes (state Θ absorbs column parities, ρ/π rotate and permute lanes, χ is the nonlinear $a_x \mapsto a_x \oplus (\overline{a_{x+1}} \wedge a_{x+2})$, ι adds a round constant).

3.4 Key and signature decoding

Base-q packing. A polynomial's n coefficients are packed K=18 per felt,

$$\mathsf{felt}_\ell \;=\; \sum_{j=0}^{K-1} v_{18\ell+j}\; q^j, \qquad q^{18} \approx 2^{244.5} < 2^{252}, \tag{8}$$

giving $\lceil n/18 \rceil = 29$ felts. Unpacking is repeated division–remainder by q; canonicality is enforced by requiring the final quotient to vanish (this is the "pk coefficients $< q$ on read" check).

Compressed signature. Falcon's "Compress" encodes each s_2 coefficient as a sign bit, 7 low bits, and a unary high part: reading t zero-bits then a one gives magnitude $|c| = (t \ll 7) \vee \text{low}$. Decoding is the inverse bit-reader; a negative zero is rejected.

3.5 Verification

Two equivalent formulations, differing in what the signer supplies.

Direct-NTT. Compute the product on-chain:

$$s_2 h = \mathrm{INTT}\big(\mathrm{NTT}(s_2) \odot \mathrm{NTT}(h)\big), \quad s_1 = c - s_2 h, \quad \text{accept iff (2).} \tag{9}$$

Hint-based. The signer additionally supplies $u = s_2 h$; the verifier checks consistency with two *forward* transforms (no inverse), using (5):

$$\mathrm{NTT}(u) \stackrel{?}{=} \mathrm{NTT}(s_2) \odot \hat{h}, \qquad s_1 = c - u, \qquad \text{accept iff (2).} \tag{10}$$

Because (5) holds for *any* valid negacyclic transform, the on-chain NTT need not match the signer's convention—only be internally consistent.

4 Verification data flow

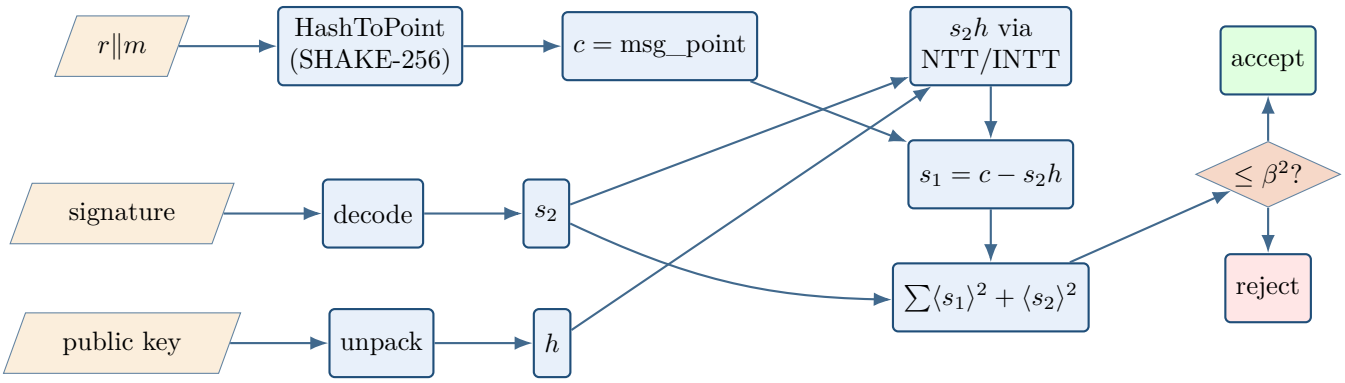

Figure 2: On-chain verification pipeline. Hash-to-point may be precomputed off-chain (FN-DSA interop path), in which case c arrives in the signature payload.

5 Code-generation pipeline

The unrolled NTT is not hand-written: a Rust circuit DSL traces the transform, validates it by replay, and emits Cairo. Figure 3 shows the flow; the same op-trace is checked against an integer reference before emission (the correctness oracle).

6 Optimization

6.1 Typed bounds and `felt252` mode

Representing coefficients as `BoundedInt`$\langle 0, q-1\rangle$ lets the compiler prove the absence of overflow, eliminating runtime range checks. When all intermediates provably stay below 2^{128}, the generator switches to native `felt252` arithmetic and reduces only where necessary—cutting the generated NTT from ∼41.6k lines (typed) to ∼11.3k.

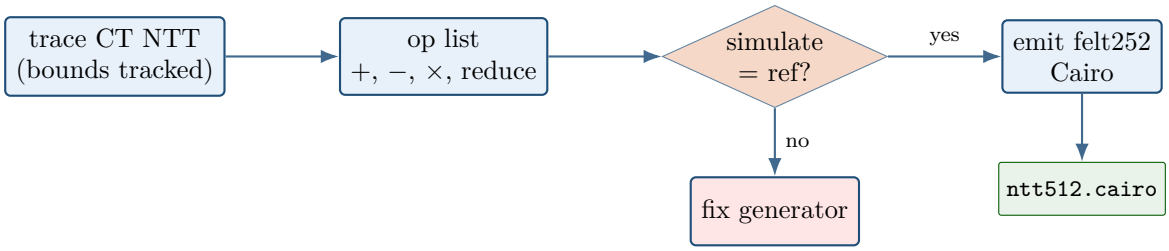

Figure 3: Code generation: bounds-tracking trace → simulate oracle → emit. The generator is a strongly-typed replacement for a Python DSL and shares constants with the Rust reference.

6.2 Lazy reduction and the shift trick

A butterfly (4) roughly multiplies the magnitude bound by q. After ℓ unreduced layers,

$$|\text{value}| \;\le\; (q-1)\,q^{\ell}. \tag{11}$$

Reducing every 6 layers keeps this below $q^7 \approx 2^{95} < 2^{128}$, so a `felt` value maps faithfully to `u128` and reduction is a single mod q. Subtraction is kept non-negative in the field by adding a shift: for $a - b$ with $b \in [0, B]$,

$$a - b \;\equiv\; a + \Delta - b \pmod{q}, \qquad \Delta = q\lceil B/q \rceil \ge B, \tag{12}$$

so the emitted expression never underflows into a large field element.

6.3 Deployability: the contract class-size cap

The fully-unrolled NTT is fast but large, and the binding constraint is not a transient compiler quirk—it is Starknet's *maximum contract class size*. A contract exposing `verify_hint_512` (both forward transforms plus the norm core; the twiddles and butterfly code are shared once via `#[inline(never)]`) compiles to

	Sierra felts	bytecode	multiple of cap
allowed (class-size cap)	81,920	4,089,446 B	—
unrolled `ntt_512` verifier	306,442	17,728,684 B	3.74× / 4.33×

so it cannot be declared on-chain; the universal-sierra-compiler additionally raises an `ApChangeError` (offset overflow) on the contract path. Emitting the transform as one function *per NTT layer*, threaded through an `Array` of `felt252` (`emit_module_layered`), produces valid Sierra with small per-frame offsets but does *not* shrink the total below the cap—the code is the same size, merely partitioned. The unrolled path is therefore best read as a cost *lower bound* and a proving-context artefact, not a deployable contract.

6.4 The deployable looped NTT

To fit the cap we express the identical transform—same twiddle table, same Cooley–Tukey structure as (4), hence *bit-for-bit* equal to the unrolled output (asserted coefficient-by-coefficient in the test suite)—as a compact loop of a few kilobytes. A naive loop costs as much as published looped verifiers (∼66 M Sierra gas); three choices recover most of the gap:

1. **`u64` end-to-end.** The working buffer is `Array<u64>`, so a butterfly never pays a field conversion; the only `felt252`↔`u64` casts are one pass at load and one at output.

2. **Nested block/index loops.** Iterating blocks then offsets removes the per-element division that a flat index loop needs to recover the block and twiddle indices—replacing range-checked `div_rem` with plain additions.

3. **Additive-growth lazy reduction.** Only the product $t = (\zeta x) \bmod q$ is reduced per butterfly; the outputs $x+t$ and $x+q-t$ accumulate *unreduced*. Each layer then grows the bound by at most $+q$ (additively, not $\times q$), so after 9 layers values stay below $10q < 2^{17}$—never overflowing `u64`, always non-negative—and a *single* final $\bmod q$ canonicalises. This trades 9 full reduction passes for one, while keeping exactly one `div_rem` per butterfly.

Because Cairo arrays are append-only, each layer reads the previous layer's `Span` and appends the next in index order (a per-block bottoms buffer preserves order in a single pass). Only the 512-entry `u64` twiddle table is generated (`--zetas` mode); the loop body is hand-written. The result, `ntt_512_looped`, is 34.4 M Sierra gas—roughly half a baseline looped NTT—and, being small, *declares and deploys*.

6.5 Hash choice

Hash-to-point dominates cost. Measured per Keccak-$f[1600]$ permutation in pure Cairo versus the Poseidon builtin:

per permutation	steps	L2 gas
Poseidon (builtin)	∼36	∼3.9k
Keccak-$f[1600]$ (pure Cairo)	∼360,000	∼50 M

Standard SHAKE-256 hash-to-point (∼10 permutations) is therefore ∼3.7 M steps—the reason a Poseidon variant is ∼50× cheaper but non-interoperable. A future `keccak_f1600` syscall (SNIP-32) would collapse the gap.

6.6 Cost-model note: K-RED is an off-chain optimization

A natural question is whether the K-RED reduction of Longa and Naehrig (ePrint 2016/504) would speed up the on-chain transform. K-RED exploits $q = 3 \cdot 2^{12} + 1$: writing a product as $C = C_0 + 2^{12} C_1$ with $C_0 = C \bmod 2^{12}$, one has $3 \cdot 2^{12} \equiv -1$, hence

$$3C \;\equiv\; 3C_0 - C_1 \pmod{q}, \tag{13}$$

and twiddles are pre-scaled by $3^{-1} \equiv 8193$ to absorb the spurious factor of 3. On a CPU this is a large win—an expensive division is replaced by a mask, a shift, a small multiply, and a subtract. *On-chain it does not apply*, because the Cairo/STARK cost model inverts the assumptions it relies on.

The unit that is paid for on-chain is VM steps plus builtin cells (`range_check`, `bitwise`), not CPU cycles. There, a field multiply is a single native step with no range check, whereas the bit operations K-RED depends on—$C \,\&\, (2^{12}-1)$ and $C \gg 12$—compile to a range-checked `div_rem` (equivalently a `bitwise` call). The cheap-on-CPU primitive is precisely the expensive-on-Cairo one:

	CPU (K-RED's target)	on-chain Cairo
field multiply	moderate	**cheap** (1 step)
shift / mask	free	**range-checked** (dear)
full reduction mod q	very expensive	one `div_rem`

The decisive point is Section 6.2: because q is tiny beside the $\sim 2^{251}$ field, we reduce *lazily*, so most butterflies carry *zero* range checks. K-RED forces a partial reduction on *every* butterfly, reintroducing one range check $\times$ 2304 that lazy reduction removes:

per butterfly	range checks	field ops
lazy felt252 (this work)	0	$\sim$3–4 (mul, add, sub)
K-RED	1 (`div_rem` by 2^{12})	+2 ($3C_0 - C_1$), output non-canonical

No reduction is even saved: in Cairo `div_rem` by q and by 2^{12} cost the same range check, so K-RED only adds the $3C_0 - C_1$ and the 3^{-1} twiddle scaling. In one line: K-RED fits a small-field machine where multiplies are dear and bit-tricks free (the native signer); the on-chain verifier is a huge-field machine where multiplies are cheap and bit-tricks dear, so lazy reduction over $\mathcal{R}_q$ is the right lever instead.

7 Measured results

Measured under scarb 2.18 / starknet-foundry 0.59 / universal-sierra-compiler 2.9. Two metrics appear below and *must not be conflated*: Cairo *steps* (reported by `scarb execute`, the modern replacement for `scarb cairo-run`) and *Sierra gas* (reported by `snforge`, the unit Starknet actually bills). The step figures are the ones comparable to published Falcon-on-Starknet step counts.

Single transform.

NTT-512	Sierra gas	deployable
unrolled (felt252, lazy reduce)	6.5 M	no (3.74× over cap)
looped (this work, §6.4)	34.4 M	**yes**
baseline looped (reference point)	$\sim$65.8 M	yes

Full verify on a real fn-dsa signature (two forward NTTs + hint/norm core; base-q unpacking included; hash-to-point excluded from this entrypoint).

path	steps	Sierra gas	deployable
unrolled (`scarb execute`)	233,488	—	no
looped (`scarb execute`)	717,820	—	**yes**
looped, deployed (`snforge`)	—	94.1 M	**yes**

The `snforge` row is a genuine on-chain measurement: the `FalconVerifier` contract is declared, deployed, and invoked, and it accepts the signature.

Self-contained verify, deployed live on Sepolia. The `verify_full` entrypoint computes the challenge on-chain—msg_point = HashToPoint(nonce ‖ SHAKE256(pk)[0:64] ‖ `0x00` ‖ `0x00` ‖ message) with interoperable SHAKE-256—then runs the looped NTT verify. It is deployed on Starknet Sepolia and was invoked in a real transaction that *accepted* a genuine fn-dsa signature:

self-contained `verify_full` (real signature, on-chain SHAKE)	value
steps (`scarb execute`)	4,716,923
range-check builtin	400,592
L2 gas (Sepolia invoke receipt)	704,797,440

The class fits the contract size cap and the transaction fits per-transaction execution limits, so a fully interoperable standard Falcon-512 verifier *is deployable and live today.* Cost is dominated by the pure-Cairo SHAKE (∼3.7 M of the ∼4.7 M steps).

Auxiliary costs.

Component (Falcon-512)	steps	Sierra gas
$\mathbb{Z}_q$ op	∼60	∼14k
SHAKE-256 hash-to-point (pure Cairo)	3,727,918	504 M
Poseidon / Blake2s / keccak-builtin hash-to-point	—	∼5–10 M
Norm acceptance bound $\lfloor \beta^2 \rfloor$		34,034,726

Reading the numbers. On raw compute the unrolled transform wins decisively—233 k steps for the full verify, well under published ∼665 k-step figures—but it is not deployable, being 3.74× over the class-size cap (§6.3). The looped verifier *is* deployable and lands at 718 k steps, on par with those figures while running as a real contract. Adding interoperable SHAKE-256 hash-to-point on-chain (`verify_full`) takes the full self-contained verify to ∼4.7 M steps / ∼705 M L2 gas—still within both the class-size cap and per-transaction limits, hence deployed and accepting real signatures on Sepolia. Hash-to-point is the dominant term and the true frontier: a builtin-hash variant is ∼50–100× cheaper but non-interoperable (and, lacking a matching signer, cannot *accept* a real signature), while a `keccak_f1600` syscall (SNIP-32) would cut the interoperable hash sharply.